



IMPLEMENTATION ROADMAP

AI Harness

Roadmap v1.1: Тикеты, Acceptance Criteria, Тесты
10 недель, 4 фазы, 28 тикетов

База: Спецификация v1.1 (гибридный подход)

Формат: Тикет → AC → Тесты → Estimation

Покрытие: Unit + Integration + E2E

Июнь 2026

Содержание

1. Общая структура

2. Фаза 0: Foundation (W1-W2)

- HARNESS-001 — validate-harness.ts
- HARNESS-002 — JSON Schema contracts
- HARNESS-003 — Templates directory
- HARNESS-004 — dependency-cruiser config
- HARNESS-005 — Validator CI pipeline

3. Фаза 1: L1 Context System (W3)

- HARNESS-006 — AGENTS.md generation
- HARNESS-007 — Instruction Modules
- HARNESS-008 — Skills Registry

4. Фаза 2: L2 + L3 (W4-W5)

- HARNESS-009 — Quality Pipeline
- HARNESS-010 — Architecture checks
- HARNESS-011 — Safety Config
- HARNESS-012 — Knowledge Base
- HARNESS-013 — Session Protocol

5. Фаза 3: Cross-cutting (W6-W7)

- HARNESS-014 — Feedback Loop
- HARNESS-015 — Circuit Breaker
- HARNESS-016 — Triage Report

6. Фаза 4: Integration (W8-W10)

- HARNESS-017 — Interactive Setup
- HARNESS-018 — CI-only mode
- HARNESS-019 — Manifest generation
- HARNESS-020 — Dogfooding
- HARNESS-021 — Documentation

7. Матрица тестового покрытия

8. Сводная таблица

1. Общая структура

Таблица 1: Структура roadmap

Фаза	Недели	Фокус	Тикетов
Phase 0: Foundation	W1-W2	Validator, Schema, Templates, dep-cruiser	5
Phase 1: L1 Context	W3	AGENTS.md, Instruction Modules, Skills Registry	3
Phase 2: L2 + L3	W4-W5	Quality Pipeline, Safety, Knowledge Base, Session Protocol	5
Phase 3: Cross-cutting	W6-W7	Feedback Loop, Circuit Breaker, Triage Report	3
Phase 4: Integration	W8-W10	Setup flow, CI mode, Dogfooding, Docs	5

Каждый тикет содержит: **Acceptance Criteria** (Given/When/Then) и **Тесты** (Unit / Integration / E2E).

Легенда приоритетов

- **P0** — Blocker, без этого не работает следующая фаза
- **P1** — Critical, core функциональность
- **P2** — Important, но можно отложить

Фаза 0: Foundation (W1-W2)

Инфраструктура: валидатор, схемы, шаблоны, CI. Всё, что нужно до начала генерации модулей.

HARNESS-001 Создать AST-based валидатор (validate-harness.ts)

PO 5d

Описание: Переписать валидатор с bash/grep на TypeScript + remark-parse AST. 4 фазы: AST Structure, Architecture (dep-cruiser), Data Contracts (ajv), Integration.

Acceptance Criteria

- **AC1:** Given AGENTS.md with heading "## Stack" — When validator runs Phase 1 — Then PASS (AST node detected, not grep)
- **AC2:** Given AGENTS.md with "## Стек" (localized) — When validator runs — Then PASS (AST value matching is flexible)
- **AC3:** Given AGENTS.md with 65 lines — When validator runs — Then FAIL with message "65 lines (max 60)"
- **AC4:** Given .dependency-cruiser.js with forbidden rules — When validator runs Phase 2 — Then PASS
- **AC5:** Given feature_list.json matching schema — When validator runs Phase 3 — Then PASS
- **AC6:** Given missing lefthook.yml — When validator runs Phase 4 — Then FAIL
- **AC7:** Given all checks pass — When validator completes — Then generates .harness/manifest.json with results array

Тесты

- **Unit:** Parse AGENTS.md with remark-parse, extract H2 headings, verify array contains expected values
- **Unit:** Count list items under "Safety" heading in AST (should be 5+)
- **Integration:** Run validator against fixture project (valid harness) — expect exit 0 + manifest.json
- **Integration:** Run validator against fixture with missing AGENTS.md — expect exit 1 + error in results
- **Integration:** Run validator against fixture with 65-line AGENTS.md — expect FAIL on line count check
- **E2E:** npx tsx validate-harness.ts in real project — completes in < 5 seconds

Зависимости

Нет (foundation тикет). Блокирует: HARNESS-005, HARNESS-017, HARNESS-018, HARNESS-019.

HARNESS-002 Создать JSON Schema контракты

P0

3d

Описание: 3 JSON Schema: feature_list.schema.json, knowledge_frontmatterschema.json, manifest.schema.json. Интеграция с ajv-cli для валидации.

Acceptance Criteria

- **AC1:** Given valid feature_list.json — When ajv-cli validates against feature_list.schema.json — Then exit 0
- **AC2:** Given feature_list.json with status "invalid_status" — When validated — Then exit 1 with enum error
- **AC3:** Given decision.md with type "decision" but no status — When validated against knowledge schema — Then exit 1 (status required for decisions)
- **AC4:** Given lesson.md with type "lesson" and severity "high" — When validated — Then exit 0
- **AC5:** All schemas published to schemas/ directory with \$id URLs

Тесты

- **Unit:** Compile each schema with Ajv, verify no compilation errors
- **Unit:** Validate 10 positive fixtures (valid data) — all pass
- **Unit:** Validate 10 negative fixtures (invalid data) — all fail with expected errors
- **Integration:** npx ajv-cli validate --schema=X --data=Y in CI pipeline

Зависимости

Нет (foundation). Блокирует: HARNESS-003, HARNESS-013.

HARNESS-003 Создать директорию templates/**P0**

3d

Описание: Verified templates для всех security-критичных и structural артефактов: AGENTS.md, lefthook.yml, ci.yml, .dependency-cruiser.js, Knowledge Base templates, feature_list.json.

Acceptance Criteria

- **AC1:** templates/AGENTS.md exists with all required ## sections as placeholders
- **AC2:** templates/lefthook.yml exists with {{LINT_CMD}}, {{TYPECHECK_CMD}}, {{TEST_CMD}} variables
- **AC3:** templates/.github/workflows/ci.yml exists with 4+ jobs including architecture
- **AC4:** templates/.dependency-cruiser.js exists with at least 2 forbidden rules
- **AC5:** templates/knowledge/ contains .template-decision.md, .template-lesson.md, .template-pattern.md
- **AC6:** templates/feature_list.json exists as valid JSON matching feature_list.schema.json

Тесты

- **Unit:** Each template file exists and is non-empty
- **Unit:** lefthook.yml template contains exactly 3 {{VARIABLE}} placeholders
- **Unit:** ci.yml template has 4 jobs (lint, typecheck, test, architecture, build)
- **Integration:** Copy all templates to temp dir, run validator — all structural checks pass
- **Integration:** feature_list.json template validates against feature_list.schema.json

Зависимости

HARNESS-002 (schema needed to validate templates). Блокирует: HARNESS-006, HARNESS-009.

HARNESS-004 Настроить dependency-cruiser конфигурацию**P1** 2d

Описание: Создать .dependency-cruiser.js с правилами для layered архитектуры. Интеграция в lefthook и CI.

Acceptance Criteria

- **AC1:** Given import from src/ui to src/data-access — When depcruise runs — Then exits with error
- **AC2:** Given import from src/services to src/data-access — When depcruise runs — Then passes
- **AC3:** Given import from src/ui to src/services/index.ts — When depcruise runs — Then passes (allowed public API)
- **AC4:** depcruise integrated as "architecture" command in lefthook.yml

Тесты

- **Unit:** Config file exports object with forbidden[] array of length ≥ 2
- **Integration:** Create fixture project with violating import — depcruise fails
- **Integration:** Create fixture project with valid imports only — depcruise passes
- **E2E:** lefthook run architecture — executes depcruise with correct config

Зависимости

HARNESS-003 (template). Блокирует: HARNESS-010.

HARNESS-005 Настроить CI pipeline для валидатора**P1** 2d

Описание: GitHub Action, который запускает validate-harness.ts на каждом push/PR. Deterministic check без LLM.

Acceptance Criteria

- **AC1:** Given push to main — When GHA triggers — Then validate-harness.ts runs and passes
- **AC2:** Given PR that breaks AGENTS.md structure — When GHA runs — Then PR blocked (check fails)
- **AC3:** CI completes in < 2 minutes
- **AC4:** Results posted as PR annotations (pass/fail per phase)

Тесты

- **Integration:** actionlint on workflow YAML — syntax valid
- **E2E:** Push to test branch — GHA triggers, validator runs, result posted
- **E2E:** Create PR with deleted AGENTS.md — GHA fails, PR blocked

Зависимости

HARNESS-001 (validator). Блокирует: нет (последний Phase 0).

Фаза 1: L1 Context System (W3)

AGENTS.md, Instruction Modules, Skills Registry. Генерация через skill, валидация через validator.

HARNESS-006 Реализовать генерацию AGENTS.md

PO 3d

Описание: Агент читает SKILL.md, сканирует проект (package.json), генерирует AGENTS.md < 50 строк. Включает раздел Architecture Rules со ссылкой на .dependency-cruiser.js.

Acceptance Criteria

- **AC1:** Given Next.js project with package.json — When agent runs skill — Then AGENTS.md generated with correct Stack section
- **AC2:** Given AGENTS.md generated — When validator runs — Then Phase 1 (AST) passes
- **AC3:** Given AGENTS.md — Then it contains "## Architecture Rules" section referencing .dependency-cruiser.js
- **AC4:** Given AGENTS.md — Then it contains "## Safety" section with 5+ forbidden operations
- **AC5:** Given AGENTS.md — Then line count < 50

Тесты

- **Integration:** Generate AGENTS.md for React project — validator AST phase passes
- **Integration:** Generate AGENTS.md for Python project — Stack section mentions Python tools
- **Integration:** Generate AGENTS.md for Go project — Commands section uses go commands
- **E2E:** Full flow: skill → generation → validation → manifest created

Зависимости

HARNESS-001, HARNESS-003. Блокирует: HARNESS-011.

HARNESS-007 Реализовать Instruction Modules**P1** 3d

Описание: Агент генерирует local/ инструкции под стек проекта. shared/ подключается через git subtree.

Acceptance Criteria

- **AC1:** Given React project — When agent runs — Then .claude/instructions/local/react-components.md created
- **AC2:** Given project with API routes — Then .claude/instructions/local/api-endpoints.md created
- **AC3:** Each .md has valid frontmatter (--- block with name, type, trigger, tags)
- **AC4:** Validator passes on Instruction Modules checks (2+ local, 1+ shared)
- **AC5:** shared/ contains SKILL.md files from registry

Тесты

- **Unit:** Frontmatter parser extracts name, type, trigger from fixture .md
- **Integration:** Generate for React project — react-components.md exists with frontmatter
- **Integration:** Generate for Vue project — vue-components.md exists (not react)
- **E2E:** git subtree add shared registry — files appear in .claude/instructions/shared/

Зависимости

HARNESS-001, HARNESS-003. Блокирует: HARNESS-008.

HARNESS-008 Создать Skills Registry (git repo)**P2** 2d

Описание: Git-репозиторий с shared-инструкциями. Semver tags. Структура: skills/<name>/SKILL.md.

Acceptance Criteria

- **AC1:** Git repo exists with at least 3 shared skills (react-component, api-endpoint, e2e-test)
- **AC2:** Repo has semver tag v1.0.0
- **AC3:** git subtree add pulls skills into .claude/instructions/shared/
- **AC4:** Each SKILL.md has valid structure (## Name, ## Setup, ## Execution)

Тесты

- **Unit:** Each SKILL.md in registry has required sections
- **Integration:** git subtree add --prefix=... registry v1.0.0 — succeeds, files present
- **Integration:** git subtree pull updates to v1.1.0 — new files appear

Зависимости

HARNESS-007. Блокирует: нет.

Фаза 2: L2 Quality & Safety + L3 Memory & Protocol (W4-W5)

Quality Pipeline с dep-cruiser, Safety Config, Knowledge Base, Session Protocol. 5 тикетов.

HARNESS-009 Реализовать Quality Pipeline (lefthook + CI)

PO 3d

Описание: Агент генерирует lefthook.yml и .github/workflows/ci.yml из templates. Заменяет {{VARIABLES}} на команды под стек.

Acceptance Criteria

- **AC1:** Given TypeScript project — When generated — Then lefthook.yml uses "biome check" for lint
- **AC2:** Given Python project — When generated — Then lefthook.yml uses "ruff check" for lint
- **AC3:** CI workflow has 4+ parallel jobs: lint, typecheck, test, architecture
- **AC4:** Validator Phase 4 passes (3+ lefthook commands, 4+ CI jobs)
- **AC5:** lefthook run pre-commit executes all commands in parallel

Тесты

- **Unit:** Template variable replacement — {{LINT_CMD}} -> correct command per stack
- **Integration:** lefthook validate — config syntax is valid
- **Integration:** actionlint on ci.yml — workflow syntax valid
- **E2E:** Push to branch — GHA runs all jobs, completes in < 3 min

Зависимости

HARNESS-003, HARNESS-004. Блокирует: HARNESS-010.

HARNESS-010 Интегрировать Architecture Checks в pipeline**P1**

2d

Описание: dependency-cruiser как "architecture" job в lefthook и CI. Forbidden rules для layered архитектуры.

Acceptance Criteria

- **AC1:** Given code with cross-layer import violation — When lefthook runs — Then architecture check fails
- **AC2:** Given code with no violations — When CI runs — Then architecture job passes
- **AC3:** Validator Phase 2 passes (.dependency-cruiser.js exists, forbidden rules defined)
- **AC4:** AGENTS.md "Architecture Rules" section references working dep-cruiser config

Тесты

- **Integration:** Fixture with src/ui importing src/data-access — depcruise fails
- **Integration:** Fixture with valid layered imports — depcruise passes
- **E2E:** Commit with violating import — lefthook blocks commit

Зависимости

HARNESS-004, HARNESS-009. Блокирует: нет.

HARNESS-011 Реализовать Safety Config (embedded)**P1**

1d

Описание: Раздел ## Safety в AGENTS.md с фиксированным списком запретов. 5+ items. Проверяется через AST (list items под heading).

Acceptance Criteria

- **AC1:** Given AGENTS.md generated by skill — Then ## Safety section has 5+ list items
- **AC2:** Safety items include: rm -rf, git push --force, .env edit, CSP change
- **AC3:** Validator counts list items under Safety heading (AST, not grep)

Тесты

- **Unit:** AST visitor counts list items under specific H2 heading
- **Integration:** AGENTS.md with 4 safety items — validator FAILS
- **Integration:** AGENTS.md with 6 safety items — validator PASSES

Зависимости

HARNESS-006. Блокирует: нет.

HARNESS-012 Реализовать Knowledge Base**P1** 2d

Описание: docs/knowledge/ с шаблонами (.template-*.md) и JSON Schema для frontmatter валидации. Агент создает записи по ходу работы.

Acceptance Criteria

- **AC1:** Given setup complete — Then docs/knowledge/ exists with 3 templates
- **AC2:** .template-decision.md has frontmatter with type, tags, date, status fields
- **AC3:** knowledge_frontmatterschema.json validates frontmatter correctly
- **AC4:** Given decision.md without status — When validated — Then fails (status required for type=decision)

Тесты

- **Unit:** Frontmatter extraction from fixture .md files
- **Unit:** Schema validation: valid decision passes, invalid decision fails
- **Integration:** Setup creates docs/knowledge/ with all 3 templates
- **Integration:** Create NNN-lesson.md with valid frontmatter — ajv validates

Зависимости

HARNESS-002, HARNESS-003. Блокирует: HARNESS-014.

HARNESS-013 Реализовать Session Protocol**P1** 2d

Описание: Раздел в AGENTS.md + feature_list.json с JSON Schema валидацией. Протокол сессии агента.

Acceptance Criteria

- **AC1:** AGENTS.md contains ## Session Protocol section
- **AC2:** feature_list.json is valid per feature_list.schema.json
- **AC3:** feature_list.json has features[] array with items having id (FXX), title, status enum
- **AC4:** Given invalid feature_list.json (status="invalid") — When ajv validates — Then fails with enum error

Тесты

- **Unit:** feature_list.schema.json compiles with Ajv without errors
- **Unit:** Validate 5 positive fixtures — all pass
- **Unit:** Validate 5 negative fixtures — all fail with expected errors
- **Integration:** Generated feature_list.json validates against schema

Зависимости

HARNESS-002, HARNESS-006. Блокирует: нет.

Фаза 3: Cross-cutting Feedback (W6-W7)

Feedback Loop, Circuit Breaker, Triage Report. Предотвращение deadlock'ов.

HARNESS-014 Реализовать Feedback Loop

P1 2d

Описание: Раздел в AGENTS.md. Цикл: ошибка -> lesson -> обновление instruction. Агент записывает в docs/knowledge/ при каждом catch pipeline'ом.

Acceptance Criteria

- **AC1:** AGENTS.md contains ## Feedback Loop section describing the cycle
- **AC2:** Given lint catches `any` usage — When agent fixes — Then lesson recorded in docs/knowledge/
- **AC3:** Lesson follows .template-lesson.md format with type, tags, date, severity
- **AC4:** Recorded lesson validates against knowledge_frontmatter.schema.json

Тесты

- **Unit:** Lesson generator creates .md from template with correct frontmatter
- **Integration:** Simulate pipeline failure -> agent records lesson -> file exists and validates
- **E2E:** Agent session with lint error -> lesson created -> next session agent reads lesson

Зависимости

HARNESS-012. Блокирует: HARNESS-015.

HARNESS-015 Реализовать Circuit Breaker**P0** 3d

Описание: ATTEMPTS_LIMIT=3. После 3 неудачных попыток починить pipeline — STOP, создание triage_report.md, передача human'у.

Acceptance Criteria

- **AC1:** AGENTS.md contains "CIRCUIT BREAKER" block with ATTEMPTS_LIMIT=3
- **AC2:** Given pipeline fails 3 times in a row — When agent tries 3rd fix — Then creates triage_report.md and stops
- **AC3:** Given pipeline fails 2 times then passes on 3rd — When 3rd attempt succeeds — Then no triage report, normal flow continues
- **AC4:** Circuit breaker state resets between independent features (F02 failure doesn't count for F03)

Тесты

- **Unit:** Circuit breaker counter increments on failure, resets on success
- **Unit:** After 3 failures — isOpen() returns true, further attempts blocked
- **Integration:** Simulate 3 pipeline failures -> triage_report.md created with correct structure
- **Integration:** 2 failures + 1 success -> no triage report, counter resets
- **E2E:** Full agent session with unfixable error -> stops after 3 attempts, human gets report

Зависимости

HARNESS-014. Блокирует: HARNESS-016.

HARNESS-016 Создать Triage Report шаблон**P1** 1d

Описание: Шаблон triage_report.md с frontmatter (type: triage, date, attempts, feature) и разделами Failure, Attempts, Analysis, Recommendation.

Acceptance Criteria

- **AC1:** template/triage_report.md exists with all required sections
- **AC2:** Generated triage_report.md has frontmatter: type, date, attempts, feature
- **AC3:** Report contains: Failure description, list of attempts, analysis, human recommendation
- **AC4:** Report validates against knowledge schema (type=triage)

Тесты

- **Unit:** Template has all 4 sections (Failure, Attempts, Analysis, Recommendation)
- **Integration:** Generate triage report from fixture data — all sections populated
- **Integration:** Report frontmatter validates against schema

Зависимости

HARNESS-015. Блокирует: нет.

Фаза 4: Integration & Polish (W8-W10)

Interactive Setup flow, CI-only mode, dogfooding, документация. 5 тикетов.

HARNESS-017 Реализовать Interactive Setup flow

PO 4d

Описание: Полный flow: агент читает SKILL.md -> сканирует проект -> задает вопросы -> генерирует все модули -> копирует templates -> запускает validator -> генерирует manifest.

Acceptance Criteria

- **AC1:** Given empty project with package.json — When agent runs skill — Then all 8 modules generated/created
- **AC2:** Agent asks 3-5 clarifying questions before generation
- **AC3:** After generation, validator runs automatically
- **AC4:** Given validator passes — Then .harness/manifest.json created and committed
- **AC5:** Given validator fails — Then agent shows errors and offers to fix (within Circuit Breaker limit)
- **AC6:** Setup completes in < 10 minutes for standard project

Тесты

- **Integration:** Setup on React project — all 8 modules created, validator passes
- **Integration:** Setup on Python project — correct tools selected (ruff, mypy)
- **Integration:** Setup on Go project — correct tools selected (golangci-lint)
- **E2E:** Full setup -> validator pass -> manifest created -> git commit
- **E2E:** Setup with invalid project (no package.json) -> graceful error handling

Зависимости

Все предыдущие тикеты (HARNESS-001..016). Блокирует: HARNESS-018, HARNESS-020.

HARNESS-018 Реализовать CI-only validation mode**P1** 2d

Описание: GitHub Action, который запускает validate-harness.ts без LLM. Deterministic gate для PR.

Acceptance Criteria

- **AC1:** Given push to main — When GHA triggers — Then validator runs without LLM
- **AC2:** Given PR that breaks harness structure — When GHA runs — Then PR blocked
- **AC3:** CI completes in < 2 minutes
- **AC4:** Results visible as PR annotations (pass/fail per phase)

Тесты

- **Integration:** actionlint on workflow YAML — syntax valid
- **E2E:** Push to test branch — GHA triggers, completes, result posted
- **E2E:** PR deleting AGENTS.md — GHA fails, merge blocked

Зависимости

HARNESS-017. Блокирует: нет.

HARNESS-019 Реализовать Manifest generation**P1** 1d

Описание: .harness/manifest.json генерируется validator'ом после успешной проверки. Lock-файл для аудита.

Acceptance Criteria

- **AC1:** Given validator passes — Then .harness/manifest.json created with results array
- **AC2:** Manifest contains: validated_at, version, errors, warnings, modules summary
- **AC3:** Manifest validates against manifest.schema.json
- **AC4:** Manifest committed with harness setup

Тесты

- **Unit:** Manifest structure matches schema
- **Integration:** Validator run -> manifest created -> validates against schema

Зависимости

HARNESS-001. Блокирует: нет.

HARNESS-020 Dogfooding: применить harness к реальному проекту

P1

5d

Описание: Применить harness к реальному проекту команды. Записать lessons. Выявить friction points.

Acceptance Criteria

- **AC1:** Harness applied to at least 1 real project (not fixture)
- **AC2:** All 8 modules functional in production context
- **AC3:** At least 3 lessons recorded in docs/knowledge/
- **AC4:** CI validation passes on real project
- **AC5:** Team feedback collected and documented

Тесты

- **E2E:** Full setup on real project -> all checks pass -> team can use harness
- **E2E:** Developer adds feature -> conventional commit -> pipeline passes
- **E2E:** Developer violates architecture rule -> depcruise catches in pre-commit

Зависимости

HARNESS-017. Блокирует: HARNESS-021.

HARNESS-021 Написать документацию и migration guide

P2

3d

Описание: README, Setup Guide, Troubleshooting, Migration from other setups.

Acceptance Criteria

- **AC1:** README.md with quick start (3 commands to setup)
- **AC2:** SETUP.md with full flow and screenshots
- **AC3:** TROUBLESHOOTING.md with common errors and fixes
- **AC4:** MIGRATION.md for projects with existing lint/setup configs

Тесты

- **E2E:** New team member follows README -> setup complete in < 15 min

Зависимости

HARNESS-020. Блокирует: нет (последний тикет).

7. Матрица тестового покрытия

Таблица 2: Покрытие по тикетам и типам тестов

Тикет	Unit	Integration	E2E	Всего
HARNESS-001 (Validator)	3	3	1	7
HARNESS-002 (Schema)	4	1	0	5
HARNESS-003 (Templates)	3	2	0	5
HARNESS-004 (Dep-cruiser)	1	2	1	4
HARNESS-005 (CI)	0	1	2	3
HARNESS-006 (AGENTS.md)	0	3	1	4
HARNESS-007 (Instructions)	1	3	1	5
HARNESS-008 (Registry)	1	2	0	3
HARNESS-009 (Quality)	1	2	1	4
HARNESS-010 (Arch checks)	0	2	1	3
HARNESS-011 (Safety)	1	2	0	3
HARNESS-012 (Knowledge)	2	2	0	4
HARNESS-013 (Session)	3	1	0	4
HARNESS-014 (Feedback)	1	1	1	3
HARNESS-015 (Circuit Breaker)	3	2	1	6
HARNESS-016 (Triage)	1	2	0	3
HARNESS-017 (Setup flow)	0	3	2	5
HARNESS-018 (CI-only)	0	1	2	3
HARNESS-019 (Manifest)	1	1	0	2
HARNESS-020 (Dogfooding)	0	0	3	3
HARNESS-021 (Docs)	0	0	1	1
ИТОГО	23	38	19	80

8. Сводная таблица

Таблица 3: Все 21 тикет: приоритет, estimation, зависимости

ID	Тикет	Prio	Est	Фаза	Блокирует
001	AST Validator	P0	5d	W1	005, 017, 018, 019
002	JSON Schema	P0	3d	W1	003, 013
003	Templates	P0	3d	W1	006, 009
004	Dep-cruiser config	P1	2d	W2	010
005	Validator CI	P1	2d	W2	-
006	AGENTS.md gen	P0	3d	W3	011
007	Instruction Modules	P1	3d	W3	008
008	Skills Registry	P2	2d	W3	-
009	Quality Pipeline	P0	3d	W4	010
010	Architecture Checks	P1	2d	W4	-
011	Safety Config	P1	1d	W4	-
012	Knowledge Base	P1	2d	W5	014
013	Session Protocol	P1	2d	W5	-
014	Feedback Loop	P1	2d	W6	015
015	Circuit Breaker	P0	3d	W6	016
016	Triage Report	P1	1d	W7	-
017	Interactive Setup	P0	4d	W8	018, 020
018	CI-only mode	P1	2d	W9	-
019	Manifest gen	P1	1d	W8	-
020	Dogfooding	P1	5d	W9	021
021	Documentation	P2	3d	W10	-

Итого

21 тикет, 46 человеко-дней, 10 недель (2 недели sprint), 80 тестов (23 unit + 38 integration + 19 e2e). Критический path: 001 -> 003 -> 006 -> 009 -> 017 -> 020 (W1 -> W3 -> W4 -> W8 -> W9).